

Password Manager

Project Documentation

A desktop password manager built in C++ with Qt Widgets, SQLite storage, and AES-256-GCM encryption.

C++17 | Qt 6 | SQLite | OpenSSL | CMake

1. Overview

Password Manager is a desktop application that stores logins, website credentials, and secure notes behind a single master password. It was built as a portfolio project to demonstrate practical, working knowledge of modern C++, object-oriented design, GUI development with Qt, relational data storage, and applied cryptography, not just theoretical familiarity with these topics.

Unlike a browser's built-in password storage or a cloud-based password manager, this application runs entirely locally. There are no accounts, no servers, and no network requests involved in storing or retrieving a credential. Everything lives in a single SQLite database file on the user's machine, and every stored password is encrypted before it touches disk.

1.1 What It Does

- Stores website/app credentials: website, URL, username, email, password, category, notes, favorite status, and timestamps.
- Protects everything behind a master password that the user creates on first launch.
- Lets the user add, edit, delete, search, filter, and organize credentials from a single dashboard.
- Generates strong random passwords on demand.
- Encrypts every stored password individually, so the raw database file is never human-readable.

1.2 Who It's For

This project is aimed at anyone who wants a simple, local, offline password manager without trusting a third-party service with their credentials, and at anyone (including the developer) who wants a concrete example of a complete C++/Qt desktop application with a real security model behind it, not just a CRUD demo.

2. Feature Breakdown

Feature	Description
Master password login	Single password unlocks the vault. First launch walks the user through creating one; it is never stored anywhere.
Dashboard	A live table of every saved credential, with a running count and status messages.
Add / edit / delete	One shared dialog handles both adding and editing a credential, with validation and a delete confirmation prompt.
Search & filter	Search by website, username, or email; filter by category; toggle favorites-only. All three combine.
Password generator	Adjustable length (8-64) and character sets, live strength indicator, cryptographically random via OpenSSL.

Copy to clipboard	Copy a username or password directly without needing to reveal it on screen first.
Show / hide passwords	Passwords are masked by default in the dashboard table; a single toggle reveals them.
Favorites & notes	Mark frequently used logins as favorites; attach free-form notes to any credential.
Change master password	Re-derives a new encryption key and re-encrypts every stored credential under it.
Auto-save	Every add, edit, or delete writes to SQLite immediately; there is no separate save step to forget.

3. Architecture

The codebase follows a simple separation of concerns: each class owns one responsibility, and the UI layer never touches SQL or cryptography directly.

Class	Responsibility
Credential	Plain data model for a single stored login (website, username, password, category, notes, timestamps, favorite flag).
DatabaseManager	Owns the SQLite connection. All reads/writes for credentials and for the master password's salt/verification hash live here.
EncryptionManager	Owns everything related to the master password: PBKDF2 key derivation, AES-256-GCM encrypt/decrypt, and password verification.
LoginDialog	Master password entry screen. Handles both first-run setup (create + confirm) and normal login (verify).
CredentialDialog	Shared add/edit form for a single credential, including inline password generation and validation.
PasswordGenerator Dialog	Standalone dialog for generating a strong random password with configurable length and character sets.
SettingsDialog	Collects current/new/confirm password fields for changing the master password.
AboutDialog	Static app information: version and tech stack.
MainWindow	The dashboard. Owns the table, search/filter state, and all CRUD actions. Wires the UI to DatabaseManager and EncryptionManager.

3.1 Folder Structure

```
include/   Header files
src/      Implementation files (mirrors include/)
resources/ Icons and stylesheets
database/ vault.db lives here at runtime (not committed to version control)
screenshots/ App screenshots for the README
docs/     Additional documentation, including this file
```

4. Security Design

The security model is the core design decision of this project, so it's documented in detail here rather than treated as an implementation detail.

4.1 The Master Password Is Never Stored

On first launch, the user's master password is run through PBKDF2-HMAC-SHA256 with 200,000 iterations and a randomly generated 16-byte salt, producing a 32-byte encryption key. A second, independent hash (derived the same way, with a distinct salt suffix) is stored as a "verification hash." On every future login, the same derivation is repeated on the entered password, and the result is compared against the stored verification hash. If they match, the password was correct, and the freshly derived key is used to decrypt credentials for that session. The master password itself is never written to disk in any form.

4.2 Credential Encryption

Every credential password is encrypted individually using AES-256-GCM, an authenticated encryption mode. A random 12-byte initialization vector (IV) is generated for each encryption operation, so encrypting the same plaintext password twice produces two different ciphertexts. The IV and GCM authentication tag are stored alongside the ciphertext (base64-encoded) in the database.

4.3 Tamper Detection

Because GCM is an authenticated mode, any modification to the stored ciphertext, whether accidental corruption or deliberate tampering, causes decryption to fail outright rather than silently returning corrupted or incorrect plaintext. This was verified directly during testing by flipping a single byte in a real ciphertext and confirming decryption correctly rejected it.

4.4 Changing the Master Password

Changing the master password re-verifies the current password against the stored verification hash, derives a brand new key and salt from the new password, re-encrypts every stored credential under the new key, and only then overwrites the stored salt and verification hash. If re-encryption fails partway through, the user is shown an explicit error rather than silently ending up in a mixed, inconsistent state.

5. How It Works

5.1 First Launch

- 1 The app checks the database for existing master password settings.
- 2 If none exist, the user is prompted to create a master password (minimum 8 characters, entered twice to confirm).
- 3 A random salt and verification hash are generated and saved. The vault is now initialized.

5.2 Every Subsequent Launch

- 1 The user is prompted for their master password.
- 2 The entered password is checked against the stored verification hash. If it fails, the user can retry.
- 3 On success, all credentials are loaded from SQLite and decrypted into memory for the session.

5.3 Adding or Editing a Credential

- 1 The user fills in the credential form (website, username, password, category, notes, etc.).
- 2 On save, the password field is encrypted with the current session key before being written to SQLite.
- 3 The plaintext version is kept only in memory, for display and future edits during the current session.

6. Tech Stack

Layer	Technology
Language	C++17
GUI Framework	Qt 6 (Widgets, Sql modules)
Local Storage	SQLite (via QSql)
Cryptography	OpenSSL (PBKDF2-HMAC-SHA256, AES-256-GCM)
Build System	CMake 3.16+
Version Control	Git / GitHub

7. Testing & Verification

In addition to compiling and running the application itself, a series of standalone test programs (not shipped with the app) were written during development to verify correctness at each phase:

- **Credential model:** validation logic and timestamp behavior.

- **CRUD logic:** add/edit/delete against both an in-memory list and real SQLite, including persistence across a simulated application restart.
- **Search & filter:** search, category filter, and favorites-only toggle tested individually and in combination.
- **Encryption correctness:** round-trip encrypt/decrypt, key re-derivation after a simulated restart, wrong-password rejection, random-IV behavior, and GCM tamper detection (a deliberately flipped ciphertext byte was confirmed to fail decryption).
- **Master password change:** verified that the old password is correctly rejected after a change, the new password works, and existing credentials still decrypt correctly after re-encryption.
- **Password generator:** confirmed length is respected, output draws only from the selected character pool, and 200 generations showed full coverage of the character pool with no bias.
- **Edge cases:** empty vault behavior, invalid/non-existent ids, empty and very long (5000-character) passwords, Unicode passwords (accents, CJK, emoji), and malformed ciphertext input.
- **Memory safety:** the full test suite was run under Valgrind, reporting zero memory leaks and zero errors across millions of allocations, including a stress test of 20 consecutive database open/close cycles.

8. Building From Source

Requirements:

- Qt 6 (Widgets and Sql modules)
- CMake 3.16 or later
- OpenSSL development headers (libssl-dev on Linux; via MSYS2 on Windows with MinGW)
- A C++17-capable compiler (MSVC, MinGW, GCC, or Clang)

Build steps:

```
mkdir build && cd build
cmake ..
cmake --build .
```

On first launch, the application prompts for a new master password (minimum 8 characters). This password cannot be recovered if forgotten; there is no recovery backdoor by design.

9. Project Roadmap

[Done] Project setup, CMake build, basic UI

[Done] Login screen

[Done] Credential data model

[Done] CRUD operations

[Done] SQLite integration

[Done] Search & filter

[Done] Encryption (AES-256-GCM + PBKDF2)

[Done] Password generator, clipboard copy, show/hide, settings, about

[Done] Testing & cleanup

[Planned] Dark mode

[Planned] Cloud sync

[Planned] Browser extension

This project is available for personal and educational use. Feel free to fork it and adapt it for your own portfolio.